

AssureCode

Autonomous AI Escrow & Verification Platform

A five-phase, cryptographically anchored system that matches clients with freelancers, verifies delivered code through a zero-trust CI/CD pipeline, guards project scope with a retrieval-augmented agent, and settles escrow autonomously through an explainable trust-scoring oracle.

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Project Objectives	4
3. The 5-Phase System Architecture	6
Phase 1 — AI Matchmaking & Cryptographic Initialization	6
Phase 2 — The Zero-Trust CI/CD Verification Engine	7
Phase 3 — Agentic AI Scope Guard (RAG)	8
Phase 4 — Telemetry Harvesting & Explainable AI (XAI)	9
Phase 5 — Algorithmic Secure Settlement	9
4. End-to-End Trust & Data Flow	11
5. Global Technology Stack	12
6. Security, Trust & Explainability Principles	14
7. Glossary of Key Terms	15

1. Executive Summary

AssureCode is an autonomous escrow and software-delivery verification platform designed to remove the two structural weaknesses of freelance software engagements: subjective, trust-based acceptance criteria and manual, delay-prone payment release. Instead of relying on a client's word or a platform's customer-support queue to decide whether work was delivered as agreed, AssureCode encodes the agreement itself into a tamper-evident cryptographic ledger at the moment a contract is formed, then continuously and mathematically measures the freelancer's actual output against that ledger throughout the engagement.

The platform is organized as a five-phase pipeline that begins the instant a client uploads a requirements document and ends the instant escrowed funds are released. Each phase produces a verifiable artifact — a hash, a test result, a similarity score, a trust score — that becomes an input to the next phase, so that by the time funds move, the release decision rests on a chain of machine-verifiable evidence rather than a single human's judgment call. The system combines four categories of technology to achieve this: natural-language matchmaking and semantic search (Sentence-BERT, Neo4j, pgvector), tamper-evident data structures (SHA-256 Merkle hash chains in PostgreSQL), zero-trust execution and static/semantic code analysis (ephemeral Docker containers, AST parsing, LLM security scanning), and explainable decision automation (an LLM-as-a-Judge evaluator and a multi-signal Node.js settlement oracle).

The result is a platform where scope is defined once and enforced automatically, where code quality and security are verified before a human ever has to ask, where scope creep is caught in the chat window before it becomes a dispute, and where payment is a deterministic consequence of verified work rather than a negotiated afterthought.

Document Purpose & Scope

This document consolidates the project's architectural objectives, its five-phase system design, and its global technology stack into a single detailed reference. It is intended for engineering, product, and stakeholder audiences who need a complete picture of how AssureCode is designed to function end to end — from the moment a client posts a requirement to the moment a freelancer is paid.

2. Project Objectives

AssureCode is built on four architectural objectives, each mapped to a distinct phase (or pair of phases) of the system. Together they form a single design principle: **every claim the system makes about the state of a project — that a requirement was understood, that code is secure, that scope has not shifted, that work deserves payment — must be backed by a reproducible, machine-verifiable artifact, not a subjective assertion.**

1

Contract & Ledger Initialization (Phase 1)

Objective	Develop an NLP-driven matchmaking system that pairs project requirements with the right talent, and irreversibly lock the agreed business logic into a tamper-proof PostgreSQL Merkle hash chain.
Why it matters	Freelance disputes are almost always disputes about <i>what was actually agreed</i> . If the acceptance criteria live only in a chat thread or a client's memory, there is no objective way to settle a disagreement later. By converting requirements into embeddings for matching and then hashing the final agreed terms into a chained ledger the moment the contract is formed, the system removes ambiguity about scope before any code is written.
How it's achieved	A Sentence-BERT model encodes the uploaded requirements PDF into vector embeddings; a Neo4j knowledge graph of freelancer skills, past performance, and availability is queried for the closest match. Once accepted, an Agentic QA LLM derives hidden unit tests from the requirements, and the finalized terms are written into a SHA-256 Merkle hash chain in PostgreSQL, making any later alteration of the agreement cryptographically detectable.
Key tech	Sentence-BERT, Neo4j, PostgreSQL, SHA-256 Merkle chains, Stripe (escrow funding).

2

Zero-Trust CI/CD Verification (Phase 2)

Objective	Design and implement a secure, ephemeral pipeline that autonomously evaluates untrusted developer code using deterministic AST parsing followed by an LLM vulnerability scan.
Why it matters	Client-side reviewers are rarely equipped to audit code quality or security, and freelancer code is, by definition, untrusted until proven otherwise. A zero-trust pipeline assumes every submission could be broken, insecure, or malicious, and verifies it inside a disposable, isolated environment before any human or downstream system relies on it.
How it's achieved	Every push triggers an ephemeral, single-use Docker container spun up via an Apache Kafka event. Inside that sandbox, the code is first subjected to a deterministic Abstract Syntax Tree audit (structural checks such as cyclomatic complexity), then executed against the hidden tests generated in Phase 1, and finally passed through a semantic LLM scan that checks for OWASP Top 10 vulnerability patterns that static analysis alone would miss.
Key tech	Apache Kafka, Docker (ephemeral containers), AST parsers (e.g., eslint-complexity), Agentic LLM security scanner.

3

Autonomous Scope Mediation (Phase 3)

Objective	Deploy a Retrieval-Augmented Generation mediator that actively monitors client-freelancer communications and prevents scope creep by mathematically anchoring chat requests to the original hashed contract.
Why it matters	Scope creep rarely arrives as a formal change order — it arrives as a casual chat message ("can you also add..."). Left unmanaged, these requests quietly expand the freelancer's workload without corresponding compensation, or they become the seed of a payment dispute. Anchoring every request back to the hashed, immutable contract gives the system an objective yardstick for what is in scope.
How it's achieved	A RAG agent retrieves the relevant clauses of the original requirements PDF (stored as vector embeddings in pgvector) and computes the cosine similarity between each incoming chat request and the hashed contract terms. Requests that fall below a similarity threshold are flagged and autonomously blocked or routed for a formal change order, rather than silently absorbed into the freelancer's workload.
Key tech	RAG architecture, PostgreSQL + pgvector, Cosine similarity scoring, Agentic LLM (Gemini/OpenAI).

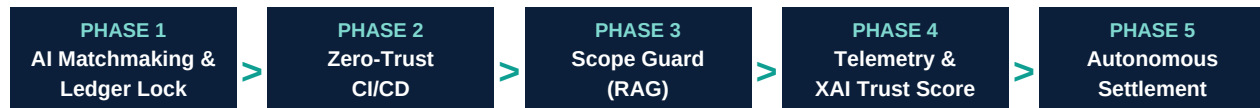
4

Objective Evaluation & Escrow Settlement (Phases 4 & 5)

Objective	Create an Explainable AI engine that generates a deterministic Trust Score from system telemetry, which autonomously triggers a Node.js oracle to release escrow funds with zero latency.
Why it matters	Even when every individual check (tests, security, scope) passes, someone still has to decide "is this milestone done and payable?" Traditionally that decision is manual, slow, and opaque. Replacing it with an explainable, deterministic score removes the delay and gives both parties a transparent, auditable justification for the payment decision.
How it's achieved	Telemetry — commit frequency, AST complexity trends, CI/CD pass rates, chat sentiment — is harvested continuously and analyzed by an LLM-as-a-Judge evaluator that outputs both a mathematical Trust Score and a human-readable justification. A Node.js multi-signal oracle then checks five independent green/red flags (AST, tests, security, scope, walkthrough video); once all five are green, it generates a cryptographically hashed invoice and calls the Stripe Connect API to route escrowed funds to the freelancer instantly.
Key tech	LLM-as-a-Judge, Neo4j (reputation update), Node.js oracle, Stripe Connect API.

3. The 5-Phase System Architecture

AssureCode operates as a continuous, autonomous pipeline. Each phase below is triggered by a concrete event (a file upload, a git push, a chat message, a milestone boundary) and produces a durable artifact that later phases depend on. The system is designed so that no phase has to "trust" the phase before it — each artifact is either cryptographically verifiable (a hash) or independently recomputable (a similarity score, a test result), which is what allows the final settlement decision in Phase 5 to be fully autonomous.



Phase 1 — AI Matchmaking & Cryptographic Initialization

Trigger: A client uploads a project requirements PDF.

Core components & actors

- **NLP Matchmaker** — Sentence-BERT model that converts requirement text into dense vector embeddings.
- **Knowledge Graph** — Neo4j graph of freelancer skills, history, and availability used to identify the optimal candidate.
- **Agentic QA LLM** — autonomously drafts hidden unit tests derived from the stated requirements, to be used later in Phase 2.
- **Ledger writer** — hashes the finalized contract terms into a SHA-256 Merkle chain.
- **Stripe Escrow** — holds the client's funds for the milestone.

Step-by-step flow

- 1 Client uploads the requirements document; text is parsed and cleaned for embedding.
- 2 Sentence-BERT encodes the requirements into a vector representation.
- 3 The Neo4j knowledge graph is queried for the freelancer whose skill/availability graph most closely matches the embedding and project constraints.
- 4 Once a match is confirmed, the Agentic QA LLM reads the requirements and autonomously generates a suite of hidden unit tests that will later validate delivered code without the freelancer knowing their exact contents in advance.
- 5 The finalized contract terms (scope, tests, price, milestones) are hashed and appended to a SHA-256 Merkle chain stored in PostgreSQL, making the agreement tamper-evident from this point forward.
- 6 The client funds the milestone into a Stripe-managed escrow account, which is held until Phase 5 settlement conditions are met.

Artifacts produced

- A vector embedding of the requirements (used later for scope similarity in Phase 3).

- A freelancer match recommendation from the Neo4j graph.
- A hidden test suite bound to the contract.
- An immutable, hash-chained contract record in PostgreSQL.
- A funded Stripe escrow account for the milestone.

Technologies used

Sentence-BERT (NLP embeddings), Neo4j (graph database), PostgreSQL (SHA-256 Merkle hash ledger), Stripe (escrow funding).

Phase 2 — The Zero-Trust CI/CD Verification Engine

Trigger: The freelancer pushes code to GitHub.

Core components & actors

- **GitHub Webhook** — detects the push event and initiates the pipeline.
- **Apache Kafka** — message broker that dispatches the verification job.
- **Ephemeral Docker container** — a disposable, isolated sandbox created fresh for this single evaluation and destroyed afterward.
- **AST parser** — performs deterministic structural analysis (e.g., cyclomatic complexity).
- **Semantic LLM security scanner** — checks for OWASP Top 10 vulnerability patterns.

Step-by-step flow

- 1 A GitHub webhook fires the moment new code is pushed to the freelancer's branch.
- 2 The webhook event is published onto an Apache Kafka topic, decoupling the trigger from the actual verification workload.
- 3 A Kafka consumer spins up a fresh, ephemeral Docker container scoped to this one evaluation — no state persists between runs, and the freelancer's code never runs on shared or persistent infrastructure.
- 4 Inside the container, an AST structural audit parses the code into its Abstract Syntax Tree and computes objective structural metrics such as cyclomatic complexity.
- 5 The hidden unit tests generated in Phase 1 are executed against the freelancer's code inside the same sandbox.
- 6 A semantic LLM scan reviews the code for security issues mapped to the OWASP Top 10 — patterns that purely structural analysis cannot catch (e.g., improper input handling, injection risk, insecure deserialization).
- 7 Results from all three checks are recorded as telemetry and the container is torn down.

Artifacts produced

- A cyclomatic-complexity / structural quality score.
- Pass/fail results for the hidden test suite.
- An OWASP-mapped security findings report.

- Immutable telemetry feeding Phase 4's Trust Score.

Technologies used

GitHub Webhooks, Apache Kafka (event broker), Docker (ephemeral containers), AST parsers (e.g., eslint-complexity), Agentic LLM (Gemini/OpenAI) for security scanning.

Phase 3 — Agentic AI Scope Guard (RAG)

Trigger: A working build exists and client-freelancer communication is active.

Core components & actors

- **Playwright** — headless browser that records a UI walkthrough video of the working application.
- **AWS S3** — stores the recorded walkthrough as a durable artifact.
- **RAG Agent** — retrieves relevant contract clauses and evaluates new chat requests against them.
- **pgvector** — vector store holding embeddings of the original hashed requirements.

Step-by-step flow

- 1 A headless Playwright browser session automatically walks through the working application's UI and records a video demonstrating current functionality.
- 2 The recording is uploaded to AWS S3 as a timestamped, milestone-linked artifact.
- 3 In parallel, the RAG agent continuously monitors the client-freelancer chat channel.
- 4 For every new request or message that looks like a feature ask, the agent retrieves the most relevant clauses of the original requirements from pgvector (which stores embeddings of the hashed Phase 1 contract).
- 5 The agent computes the cosine similarity between the new request and the retrieved contract clauses.
- 6 Requests below the similarity threshold are flagged as out-of-scope and are autonomously blocked or redirected to a formal change-order flow, rather than silently added to the freelancer's obligations.

Artifacts produced

- A timestamped UI walkthrough video stored in S3 (one of the five settlement signals in Phase 5).
- A running log of in-scope vs. flagged out-of-scope chat requests.
- A cosine-similarity audit trail tying every accepted request back to the original contract.

Technologies used

Playwright (headless UI testing/recording), AWS S3 (artifact storage), PostgreSQL + pgvector (vector memory for RAG), Agentic LLM (Gemini/OpenAI).

Phase 4 — Telemetry Harvesting & Explainable AI (XAI)

Trigger: Continuous, silent collection across the lifetime of a milestone.

Core components & actors

- **Telemetry harvester** — silently aggregates signals from every earlier phase.
- **LLM-as-a-Judge Evaluator Agent** — analyzes aggregated telemetry to produce a Trust Score.
- **Neo4j** — freelancer reputation graph updated with the new score.

Step-by-step flow

- 1 The system silently harvests JSON telemetry throughout the milestone: Git commit frequency and cadence, AST complexity scores from Phase 2, CI/CD pass rates, and chat sentiment extracted from the Phase 3 communication stream.
- 2 This telemetry is compiled into a structured JSON record per milestone.
- 3 An LLM-as-a-Judge Evaluator Agent analyzes the compiled telemetry and outputs two things: a deterministic, mathematical Trust Score, and a human-readable justification explaining exactly which signals drove that score.
- 4 The freelancer's Neo4j graph node is updated with the new Trust Score, feeding future matchmaking decisions in Phase 1 for subsequent contracts.

Artifacts produced

- A structured JSON telemetry record per milestone.
- A deterministic, explainable Trust Score.
- A human-readable justification for the score (the "X" in XAI).
- An updated freelancer reputation node in Neo4j.

Technologies used

JSON telemetry pipeline, LLM-as-a-Judge (Agentic LLM), Neo4j (reputation graph).

Phase 5 — Algorithmic Secure Settlement

Trigger: All upstream phases have reported their signals for the milestone.

Core components & actors

- **Node.js Multi-Signal Oracle** — the deterministic decision-maker that authorizes payment release.
- **Stripe Connect API** — executes the actual fund transfer to the freelancer.

Step-by-step flow

- 1 A Node.js Multi-Signal Oracle checks five independent signals for the milestone: the AST structural audit (Phase 2), the hidden test results (Phase 2), the security scan (Phase 2), the scope-compliance record (Phase 3), and the UI walkthrough video (Phase 3).
- 2 Each signal must independently report "green" — there is no single point of override; a failure in any one signal blocks settlement.
- 3 Once all five flags are green, the backend generates a cryptographically hashed invoice tied to the Phase 1 Merkle-chained contract record.
- 4 The Stripe Connect API is triggered to instantly route the escrowed funds from the client's escrow account to the freelancer, with no manual approval step in the critical path.

Artifacts produced

- A five-signal settlement audit record (AST, Tests, Security, Scope, Video).
- A cryptographically hashed, contract-linked invoice.
- A completed Stripe Connect fund transfer to the freelancer.

Technologies used

Node.js (oracle logic), Stripe Connect API (fund settlement), PostgreSQL (hash-chained invoice record).

4. End-to-End Trust & Data Flow

The five phases are designed to compound trust rather than simply hand off data. The value of this design is easiest to see by tracing a single piece of information — the original requirements document — through the entire lifecycle.

- **Origination:** the requirements PDF is embedded once by Sentence-BERT and its finalized terms are hashed once into the Phase 1 Merkle chain. From this point forward, the contract cannot be silently edited by either party without breaking the hash chain.
- **Verification anchor:** the hidden tests generated alongside that same contract in Phase 1 become the objective pass/fail bar that Phase 2's CI/CD pipeline applies to every code push — no human has to re-interpret the requirements to judge the code.
- **Scope anchor:** the same requirements embedding is reused in Phase 3, where every new chat request is measured by cosine similarity against it, so "is this in scope" is answered mathematically rather than by whoever argues more persuasively in the chat.
- **Aggregation:** Phase 4 does not re-derive trust from scratch; it aggregates the objective outputs of Phases 1–3 (test results, complexity scores, scope compliance, sentiment) into a single explainable score.
- **Resolution:** Phase 5 does not re-evaluate anything — it purely checks that the five upstream signals are green and then executes payment. By the time money moves, every fact behind the decision was established earlier in the pipeline and is independently auditable.

This is the core architectural insight of AssureCode: **trust is established once, at contract formation, and is never re-litigated — only measured against.** Every later phase either verifies compliance with the Phase 1 ledger or contributes a new, independently verifiable signal to it. No phase is asked to trust the output of a human's subjective judgment.

5. Global Technology Stack

The stack is organized around a clear separation of concerns: a conventional web application layer for client/freelancer interaction, a polyglot backend that splits transactional API logic from AI/NLP workloads, an event-driven core for decoupling triggers from heavy verification work, three purpose-specific databases (relational ledger, graph, vector), and a set of external services that handle the specialized jobs (headless browsing, artifact storage, payment settlement).

5.1 Frontend

Component	Role in AssureCode
React.js / Next.js	Client and freelancer dashboards, contract review UI, escrow status, chat interface, and the milestone/Trust Score views surfaced from Phase 4.

5.2 Backend Services

Component	Role in AssureCode
Node.js (API Gateway & Oracles)	Serves the public API layer and hosts the Phase 5 Multi-Signal Oracle that evaluates settlement conditions and calls Stripe Connect.
Python (FastAPI)	Hosts the AI/NLP services: the Sentence-BERT matchmaking service, the RAG scope-guard agent, and the LLM-as-a-Judge evaluator.

5.3 Event Broker

Component	Role in AssureCode
Apache Kafka	Decouples GitHub push events from the Phase 2 verification workload, allowing the ephemeral Docker/CI pipeline to scale independently and process pushes asynchronously.

5.4 Databases

Component	Role in AssureCode
PostgreSQL	Core relational data store and the SHA-256 tamper-evident Merkle hash ledger for contract terms and settlement invoices.
Neo4j	Knowledge graph for talent matchmaking (Phase 1) and freelancer reputation/Trust Score updates (Phase 4).
PostgreSQL + pgvector	Vector memory for the Phase 3 RAG scope guard, storing embeddings of the hashed original requirements for cosine-similarity comparison.

5.5 DevSecOps Pipeline

Component	Role in AssureCode
GitHub Webhooks	Triggers the Phase 2 verification pipeline on every freelancer code push.
Docker (Ephemeral Containers)	Provides a disposable, zero-trust execution sandbox for every single code evaluation.
AST Parsers (e.g., eslint-complexity)	Deterministic structural code analysis, including cyclomatic complexity, prior to semantic LLM review.

5.6 AI Models

Component	Role in AssureCode
Sentence-BERT	Converts natural-language requirements into vector embeddings for freelancer matchmaking and later scope-similarity comparisons.
Agentic LLMs (Gemini / OpenAI)	Power the Agentic QA test generator (Phase 1), the semantic security scanner (Phase 2), the RAG scope-guard agent (Phase 3), and the LLM-as-a-Judge Trust Score evaluator (Phase 4).

5.7 External APIs

Component	Role in AssureCode
Playwright	Headless browser automation that records UI walkthrough videos of the working application as a Phase 3/5 settlement signal.
AWS S3	Durable storage for recorded walkthrough videos and other milestone artifacts.
Stripe Connect API	Handles escrow funding at Phase 1 and instant, autonomous fund settlement to the freelancer at Phase 5.

6. Security, Trust & Explainability Principles

6.1 Tamper-Evidence by Construction

Contract terms are never stored as a plain, mutable row. Once written into the Phase 1 SHA-256 Merkle hash chain, any later modification to the contract changes its hash and breaks the chain's internal consistency, making tampering mathematically detectable rather than merely against policy.

6.2 Zero-Trust Execution

No freelancer-submitted code is ever assumed safe. Every push is executed inside a single-use, isolated Docker container that is destroyed after evaluation, and is subjected to both deterministic structural analysis (AST) and semantic security review (LLM/OWASP) before its results can influence the Trust Score or settlement decision.

6.3 Mathematical Scope Anchoring

Scope disputes are resolved by cosine similarity against the original hashed requirements, not by re-litigating intent in a chat thread. This gives both client and freelancer a consistent, reproducible standard for what was — and was not — agreed to.

6.4 Explainability (XAI)

The Trust Score produced in Phase 4 is never a black-box number. The LLM-as-a-Judge evaluator is required to produce a human-readable justification alongside the score, so that both parties can see exactly which signals (commit cadence, complexity, test pass rate, sentiment) drove the outcome.

6.5 Deterministic, Multi-Signal Settlement

Payment release in Phase 5 is never triggered by a single metric or a single approver. The Multi-Signal Oracle requires five independent green flags (AST, Tests, Security, Scope, Video) before funds move, so a weakness in any one dimension cannot be masked by strength in another.

7. Glossary of Key Terms

Merkle Hash Chain — A cryptographic data structure in which each record's hash incorporates the hash of the record before it, so altering any past entry invalidates every hash that follows, making tampering detectable.

Abstract Syntax Tree (AST) — A tree representation of a program's source code structure, used here to compute deterministic, objective metrics such as cyclomatic complexity.

Cyclomatic Complexity — A quantitative measure of the number of independent paths through a program's code, used as a proxy for maintainability and testability.

Retrieval-Augmented Generation (RAG) — An architecture in which a language model's output is grounded by first retrieving relevant stored documents (here, the hashed contract embeddings) before generating a response or decision.

Cosine Similarity — A measure of how similar two vectors are, based on the angle between them; used to compare a new chat request's embedding against the original contract's embedding.

pgvector — A PostgreSQL extension that adds native vector storage and similarity search, used as the RAG agent's vector memory.

Zero-Trust (architecture) — A security model that assumes no request, user, or piece of code is inherently trustworthy and requires explicit verification at every step, regardless of origin.

Explainable AI (XAI) — AI systems designed so that their outputs are accompanied by a human-understandable rationale, rather than being an opaque score or decision.

LLM-as-a-Judge — A pattern in which a large language model is used to evaluate and score other data or outputs (here, project telemetry) against defined criteria.

Escrow — Funds held by a neutral third party (here, via Stripe) until predefined conditions are met, at which point they are released to the intended recipient.

Oracle (settlement) — A trusted process that determines when real-world/off-chain conditions are satisfied and triggers a corresponding action — here, the Node.js service that checks all five settlement signals and authorizes fund release.

OWASP Top 10 — A widely referenced list of the most critical web application security risks, used here as the checklist for the Phase 2 semantic LLM security scan.